

Efficient Packaging and Deployment of AI Models for Edge Inference

Jaiveer Bassi

College of Science, Engineering, and Technology
Grand Canyon University, Phoenix, Arizona, USA
jaiveerbassi@yahoo.com

Abstract—Edge deployment requires more than a compact neural-network artifact: the model, runtime, service boundary, and resource policy must be reproducible as one system. This paper presents an open implementation that combines MobileNetV2, post-training integer quantization, optional magnitude pruning, a FastAPI inference service, and Docker packaging. We describe the pipeline, its operational interfaces, and a reproducibility-oriented benchmark protocol. The repository contains a reference benchmark profile reporting a reduction in model storage from 13.6MB to 3.3MB and mean inference latency from 220ms to 95ms; these values are preserved as reference/emulated results rather than presented as newly verified measurements. Energy values are derived from latency and an assumed power profile. We therefore separate measured, derived, and emulated quantities and identify the calibration, hardware, and dataset artifacts required for a publishable empirical evaluation.

Index Terms—edge inference, post-training quantization, pruning, TensorFlow Lite, FastAPI, Docker, reproducibility

I. INTRODUCTION

Running inference near the source of data can reduce network dependence, response time, and the need to transmit sensitive inputs. The same setting exposes constraints that are less visible in cloud deployment: limited memory, modest CPUs, heterogeneous accelerators, thermal budgets, and difficult software maintenance. A useful edge system must therefore treat model compression and packaging as a joint engineering problem.

Post-training quantization maps floating-point tensors to lower-precision representations and can reduce storage and arithmetic cost [1]. Pruning can remove low-contribution parameters, although the benefit depends on whether the target runtime exploits the resulting sparsity [2], [3]. MobileNetV2 is a representative compact convolutional architecture for this setting [4]. TensorFlow Lite provides an integer-oriented deployment path for embedded inference [5].

This work contributes an artifact-centered implementation that connects those model transformations to a FastAPI service and a resource-constrained Docker deployment. The repository includes source code, configuration, benchmark data, and a multi-file IEEEtran manuscript. Rather than claiming unverified hardware experiments, we make the provenance of the supplied numbers explicit. The reference profile is useful for exercising the reporting and deployment workflow, but it is not a substitute for a controlled experiment on named hardware.

The paper addresses three questions: (1) how should a compact model be converted and served at the edge, (2) which quantities can be reproduced directly from the repository, and (3) which claims require additional hardware, dataset, and measurement artifacts? Section II describes the implementation. Section III reports the reference profile and its limitations. Section IV summarizes the resulting publication and reproducibility requirements.

II. METHODOLOGY

The repository implements four layers: model preparation, integer conversion, HTTP serving, and container packaging. The intended path is MobileNetV2 with a $224 \times 224 \times 3$ input, conversion to a fully integer TensorFlow Lite model, and single-image inference through a FastAPI endpoint.

A. Model preparation and quantization

The baseline factory constructs ImageNet-pretrained MobileNetV2. The optional pruning module configures unstructured magnitude pruning with TensorFlow Model Optimization’s polynomial schedule from zero to 30% target sparsity over 1,000 steps. This operation masks individual weights; it does not remove entire channels. A training loop with pruning callbacks and evaluation is required before stripping the wrappers. Such a trained pruning artifact is not included, and neither achieved sparsity nor pruning speedup is established here.

For post-training quantization, the converter is configured with `Optimize.DEFAULT`, the built-in INT8 operator set, and INT8 input and output tensors. A representative calibration set is required. Random arrays are not accepted as evidence of calibration quality; the conversion scripts require an image directory through `EDGE_CALIBRATION_DIR`. Each image is decoded, converted to RGB, resized to 224×224 , and normalized using MobileNetV2 preprocessing before being yielded to the converter. The affine mapping is

$$q = \text{clip}(\text{round}(x/S) + Z, q_{\min}, q_{\max}), \quad (1)$$

where S and Z are the scale and zero point determined by the converter; dequantization uses $\tilde{x} = S(q - Z)$.

B. Serving and packaging

The service exposes `/health`, `/metadata`, and `/predict`. Input preprocessing reads the tensor’s actual quantization

TABLE I
REFERENCE RASPBERRY PI 4 PROFILE SUPPLIED WITH THE REPOSITORY.

Metric	FP32 reference	INT8 reference
Mean latency (ms)	220.0	95.0
90th-percentile latency (ms)	245.0	105.0
Throughput (inferences/s)	4.55	10.53
Process memory (MB)	240.0	65.0
Model size (MB)	13.6	3.3
Assumed power (W)	3.2	3.0
Derived energy (J/inference)	0.704	0.285

parameters rather than assuming that all INT8 models use the same offset. Output scores are dequantized when metadata are available. If no model is loaded, the service reports a degraded health state and returns an error from `/predict`; it does not silently fabricate predictions. This distinction is important for deployment monitoring and for honest benchmark collection.

Docker packages the service with the standalone TFLite runtime, Pillow, NumPy, and the web server. Docker Compose provides a model volume and CPU/memory limits. These limits are deployment policy, not measurements: their effect must be evaluated on the target device using repeated runs, warm-up rules, and a declared measurement tool.

C. Reproducibility protocol

For a publishable evaluation, the benchmark should record the exact model hash, TensorFlow/TFLite runtime version, operating-system image, CPU governor, thread count, input set, warm-up count, number of timed repetitions, mean and percentile definitions, and power-meter model. Accuracy must be evaluated against a fixed labeled validation set; latency and memory must be measured separately for native inference and HTTP serving. Derived energy is computed as $E = Pt$ only when P is a measured average power over the corresponding interval.

III. REFERENCE RESULTS AND LIMITATIONS

The repository includes `benchmark_results.json`, which stores a reference Raspberry Pi 4 profile. The benchmark script uses those values only when emulation is explicitly requested; failed live runs do not generate replacement results. Because the repository does not include a device log, power trace, validation predictions, or calibration images, Table I is reported as a reference/emulated profile. It must not be interpreted as an independently reproduced physical experiment.

Relative to the supplied FP32 reference, the INT8 profile corresponds to a $2.32\times$ lower mean latency, a $4.12\times$ smaller model artifact, and a $3.69\times$ lower reported process-memory figure. The energy values follow directly from the listed latency and assumed power: $0.220\text{ s} \times 3.2\text{ W} = 0.704\text{ J}$ and $0.095\text{ s} \times 3.0\text{ W} = 0.285\text{ J}$. They are therefore derived estimates, not power measurements.

The repository does not currently support a defensible claim about ImageNet accuracy: the conversion scripts previously used random calibration arrays, no validation dataset is distributed, and the original accuracy fields were inherited

reference values. Similarly, the Jetson Nano and Google Coral rows present in the earlier draft were not accompanied by raw logs or executable platform-specific benchmark code and have been removed from the manuscript. The same applies to claims about batching, concurrent multi-model execution, and container overhead. These are worthwhile future experiments, but they are not established by the current artifact.

This separation improves scientific usability. A reader can exercise conversion and serving after supplying the required dependencies and calibration data, while the missing evidence is visible instead of being hidden behind precise-looking numbers. The repository’s benchmark tool should be run in live mode only after the model and calibration data have been supplied; its output should then be archived with the environment and hardware metadata listed in Section II.

IV. CONCLUSION

The implementation connects MobileNetV2 conversion, optional pruning, TFLite inference, FastAPI serving, and Docker packaging. Its reference profile provides an illustrative comparison; it does not establish physical performance, accuracy preservation, or accelerator speedup.

An empirical submission requires a documented calibration and validation set, model and dataset hashes, raw timing and memory logs from named devices, synchronized power measurements, and a controlled container-overhead experiment. Pruning must be evaluated after training with the appropriate callbacks. The current live benchmark is a host diagnostic using synthetic inputs; it compares different backends and retains both models in one process, so it cannot isolate the effects of precision or model memory. These limitations define the next experimental steps for validating the deployment pipeline.

REFERENCES

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, M. Andreetto, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [2] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.
- [3] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, “Pruning filters for efficient convnets,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [5] Google AI Edge, “Post-training integer quantization,” https://www.tensorflow.org/lite/performance/post_training_integer_quant, accessed: September 13, 2026.